
ODI Demo Builder — Session Guide

Both of you run this simultaneously · One person shares screen on the Meet so you both see results · ~3 hours · No skill install required — paste prompts directly into Claude Code

How this works. Each phase has a prompt in a red-bordered box. Both of you paste it into your own Claude Code session at the same time. One of you shares your screen so you can compare what Claude Code produces and talk through it. When Claude Code asks a question, answer it together. When something breaks, tell Claude Code exactly what happened and ask it to fix the issue — that is the right approach for every error you hit in this session.

Before you start

5 minutes

Both of you — run this verification block before touching Claude Code. Every line should return output, not an error.

```
claude whoami
op whoami
aws sts get-caller-identity --query Account --output text
echo "Fivetran key: ${FIVETRAN_API_KEY:0:8}..."
echo "Fivetran secret: ${FIVETRAN_API_SECRET:0:8}..."
```

If anything fails, paste the error into the Meet chat and ask Claude Code: "This command failed — how do I fix it?" Claude Code will give you the exact steps.

One of you — share your screen on the Meet now. The person who shares screen is the reference point. When your outputs differ, the shared screen is what you discuss. Either person can share; just pick one and keep it consistent through the session.

Both of you — agree on these before pasting any phase prompt:

1. **Industry vertical and fictional company name** — the name must not be a real company. Claude Code will validate it.
2. **Buyer persona** — CDO, CIO, or Head of Data Engineering. This shapes the frontend copy.
3. **The exact 1Password item name for the SQL Server credentials** — Claude Code will use `op read` to fetch host, port, username, password from it.

When something breaks, do this: Paste the full error output into Claude Code and say "this failed — what is the fix?" Do not try to solve errors by editing files or commands yourself before asking Claude Code. Claude Code has full context of what it built and can diagnose faster than manual investigation. This is true for every error in every phase.

Phase 1 — Gather

target: 15 minutes

PHASE 1 — GATHER

Claude Code collects the demo context, pulls SQL Server credentials from 1Password, tests connectivity, lists tables, and gets your update-method decision.

PASTE INTO CLAUDE CODE – BOTH OF YOU

You are facilitating a hands-on Fivetran ODI demo build session. Your job is to guide the build of a production-ready ODI demo from scratch – SQL Server as the source, Fivetran as the ingest layer, Apache Iceberg on S3 as the lake, dbt for bronze/silver/gold transforms, and a React frontend as the demo surface.

Narrate your reasoning at every decision point.

Start by collecting five inputs:

1. Industry vertical and fictional company name – validate that the name does not collide with a real entity before proceeding
2. Buyer persona: CDO, CIO, or Head of Data Engineering
3. The 1Password item name for SQL Server credentials – you will fetch host, port, username, and password using `op read`
4. AWS region (default `us-east-1`) and S3 bucket name prefix
5. Target database and table selection – list these after connectivity is confirmed

Ask for all five now. Do not start any infrastructure work until you have them.

Answer Claude Code's questions. When it asks for the 1Password item name, give it the exact name from your vault. Claude Code will run `op read` to pull credentials – nothing gets pasted in plaintext.

When Claude Code asks about the update method, it will present two options: CDC (log-based, requires SQL Server Agent and `VIEW SERVER STATE` permission) and full reload. Choose the one that fits the SQL Server environment you are working with. Claude Code will confirm what it needs from the server for whichever you pick.

Phase 1 done when: connectivity confirmed, tables selected, update method decided.

Phase 2 – Provision

target: 45 minutes

PHASE 2 – PROVISION

Before pasting the Phase 2 prompt, the person sharing screen: go to fivetran.com, start creating a new destination, choose Managed Data Lake (MDLS). It shows two values – a Fivetran AWS account ID and an External ID. Share those values with each other. Both of you will need them for Terraform.

PASTE INTO CLAUDE CODE – BOTH OF YOU

Phase 1 is complete. Run Phase 2: Provision.

In order:

1. Create the project directory at `/Users/[your-username]/Documents/GitHub/[DEMO_SLUG]-odi-demo/infra/` and initialize a git repo
2. Write `main.tf` with: S3 bucket (versioning enabled, AES256 encryption, public access blocked, 30-day lifecycle to `STANDARD_IA`), three Glue databases (bronze, silver, gold), IAM role for Fivetran

MDLS with trust policy using `fivecran_aws_account_id` and `fivecran_external_id` as variables, Athena workgroup, and outputs for `bucket_name`, `fivecran_role_arn`, `bronze_db`, `silver_db`, `gold_db`

3. Write `terraform.tfvars` with placeholders for the two Fivetran values. Pause and ask me to fill them in before running `apply`
4. After I confirm the values are in `tfvars`, run `terraform init` and `terraform plan`. Print the plan and wait for my approval before applying
5. Run `terraform apply` after I approve. Capture outputs into environment variables
6. Create the Fivetran MDLS destination via POST `https://api.fivetran.com/v1/destinations` using `FIVETRAN_API_KEY` and `FIVETRAN_API_SECRET`
7. Create the SQL Server connector via POST `https://api.fivetran.com/v1/connectors` using the `1Password` credentials and the update method from Phase 1. Print and store the connector ID and deep link URL
8. Test the connector: POST `/v1/connectors/[ID]/test`. Confirm `setup_state` is `CONNECTED` before moving to Phase 3

Show me the Terraform plan before applying. Do not create the connector until the destination is confirmed.

Claude Code will pause twice and wait for your input: once when `tfvars` needs the Fivetran values, and once when it shows you the Terraform plan. Review the plan — you should see an S3 bucket, three Glue databases, an IAM role, and an Athena workgroup. Tell Claude Code "approved, apply it" when you are ready.

Phase 2 done when: Terraform applied, MDLS destination created, SQL Server connector `CONNECTED`.

If the connector test returns `INCOMPLETE`: Paste the full error text to Claude Code and ask it to diagnose. It will tell you exactly what is wrong and how to fix it.

Phase 3 — Build

target: 60 minutes

PHASE 3 — BUILD

PASTE INTO CLAUDE CODE — BOTH OF YOU

Phase 2 is complete. Run Phase 3: Build.

In order:

1. Scaffold the dbt project in `[DEMO_DIR]/dbt/`: directories `models/bronze`, `models/silver`, `models/gold`, `seeds`, `macros`, `tests`
2. Write `dbt_project.yml`: project name from demo slug, Athena profile, silver materialized as views, gold as Iceberg tables (parquet, partitioned), `store_failures` enabled
3. Write `packages.yml` with `dbt_utils` and `dbt_expectations`
4. Write `profiles.yml` using environment variables for all connection details — nothing hardcoded. Use `awsdatacatalog` as the database value
5. Generate `sources.yml` in `models/bronze/` for every table from Phase 1. Each table gets a `_fivetran_synced` not_null test and a `_fivetran_deleted` column documented
6. For each source table: ask me for the primary key column, date columns, and metric/amount columns, then generate a staging model in `models/silver/` using the standard source → renamed CTE pattern
7. Ask me to identify the main dimension entity and main fact grain, then generate at least one gold dim model and one gold fct model using Iceberg materialization
8. Apply content rules to everything generated:
 - No raw table or schema names in any string that renders in the UI

- Fictional org name used consistently in all copy
 - pipeline.json must include connector ID and full deep link URL for every connector
9. Generate the React frontend shell: App.tsx with HashRouter, Landing page with org name and buyer-persona copy, Pipeline page with connector cards that include working Open-in-Fivetran links, Analysis page with a Recharts chart placeholder
 10. Run: `dbt deps && dbt seed && dbt build --select bronze silver gold`
 11. Report results. If anything fails, paste the error and diagnose before suggesting a fix

Ask me about primary keys and grain before writing any SQL. Show me the model file list before running the build.

Claude Code will ask you questions about column structure before generating SQL. Answer them — this is where your knowledge of the source data matters. When it shows you the list of model files it plans to create, review it before telling it to proceed.

When Claude Code applies the content rules, both of you review the output together on the shared screen. Three things to confirm: no raw schema or table names visible in any UI string, the fictional company name is consistent everywhere, and each connector entry in pipeline.json has a `deep_link` field with a real fivetran.com URL.

Phase 3 done when: dbt build passes, React shell generated, content rules confirmed.

Phase 4 — Run

target: 30 minutes

PHASE 4 — RUN

PASTE INTO CLAUDE CODE — BOTH OF YOU

Phase 3 is complete. Run Phase 4: Run.

In order:

1. Trigger the first Fivetran sync: `POST /v1/connectors/[CONNECTOR_ID]/sync`
2. Poll sync status every 30 seconds and print each update until `sync_state` is `SYNCED` or `ERROR`
3. If `ERROR`, print the Fivetran error message and stop — do not run dbt on errored data
4. When `SYNCED`, verify data in Iceberg: list S3 objects in the bronze prefix for metadata.json files, then run Athena `COUNT(*)` on each source table and print row counts
5. Run `dbt build --select silver gold` against the synced data
6. Walk through the demo tour:
 - Landing page: org name visible, no raw table names, buyer persona copy correct
 - Pipeline page: SQL Server connector card visible, Open-in-Fivetran link opens the correct connector
 - Fivetran dashboard at the connector deep link: confirm `SYNCED` status
 - Athena: `SELECT * LIMIT 5` on each gold table, confirm rows returned
 - Analysis page: chart component renders
7. Print the final build summary: demo slug, Fivetran group ID, connector ID and deep link, S3 bucket, Glue database names, dbt project name, update method, sync status, count of gold models built
8. Print next steps: push repo to GitHub, add to demo catalog, run demo-blitz for a full content and design pass, add analytical charts once data volume warrants

Poll every 30 seconds. The first sync may take 5 to 30 minutes depending on table size.

Use the sync wait time to review the dbt models and React frontend that Phase 3 generated. Claude Code will keep printing status updates in the background.

Phase 4 done when: sync SYNCED, Athena row counts confirmed, dbt gold models populated, demo tour complete, summary printed.

When something breaks — how to handle it

This applies to every phase. The same approach works every time:

1. Do not try to fix the error by editing files or rerunning commands yourself first.
2. Copy the full error output — the complete text, not just the last line.
3. Paste it into Claude Code and say: **"This failed. Here is the error. What is the fix?"**
4. Claude Code will diagnose it and give you the exact corrective steps.

Common situations and what they usually mean:

WHAT YOU SEE	WHAT TO PASTE TO CLAUDE CODE
<code>op read</code> returns nothing	The full terminal output plus: "op read returned empty, here is what I got"
SQL Server port not reachable	The nc or sqlcmd output plus: "connectivity check failed, this is the result"
Terraform plan has unexpected resources	The full plan output plus: "I see resources I did not expect, please explain"
Fivetran API returns an error code	The full API response JSON plus: "the API returned an error"
dbt build fails	The full dbt output including the error block plus: "dbt build failed"
Athena query fails	The full error message plus: "Athena returned this error"

Hybrid deployment — the data-residency answer

Claude Code will surface this during Phase 2 as a short aside. It is the answer to the customer objection "we cannot send our data to a third-party cloud."

Fivetran Hybrid Deployment runs the data-movement agent inside the customer's own network. Fivetran's cloud plane receives only orchestration metadata — row counts, schema, sync timestamps. No customer data transits Fivetran's infrastructure.

What stays identical: the connector configuration, the MDLS destination, the dbt pipeline, the React frontend. Hybrid deployment is a deployment mode applied in front of the same architecture you built today.